

POS SERVICE

Documento de Requerimientos del Sistema

Version 1.0 - Julio 2026

Documento generado automaticamente a partir del analisis del repositorio POS SERVICE (backend Spring Boot + frontend Angular 20).

Indice

1. Introduccion y proposito
2. Arquitectura general
3. Requerimientos funcionales - Autenticacion
4. Requerimientos funcionales - Punto de venta
5. Requerimientos funcionales - Gestion de caja
6. Requerimientos funcionales - Movimientos de caja
7. Requerimientos funcionales - Catalogo
8. Requerimientos funcionales - Clientes
9. Requerimientos funcionales - Ordenes y reportes
10. Requerimientos funcionales - Cuentas de cajero
11. Reglas de negocio criticas
12. Requerimientos no funcionales
13. Stack tecnologico y limitaciones

1. Introduccion y proposito

Vision general

POS SERVICE es un sistema de punto de venta (POS) diseñado para gestionar operaciones comerciales diarias: registro de ventas, control de caja, administracion de catalogo de productos y categorias, gestion de clientes y generacion de reportes. El sistema esta orientado a pequenos y medianos negocios que necesitan una solucion web moderna, desplegable con Docker y accesible desde un navegador.

Usuarios del sistema

Operadores de caja: realizan ventas, registran movimientos de efectivo y consultan el historial de ordenes del turno.

Administradores o encargados: configuran el catalogo (productos y categorias), gestionan clientes, abren y cierran caja, revisan reportes y administran cuentas de cajero.

Usuarios de acceso web: se autentican con credenciales JWT (login/registro) para acceder a todas las funcionalidades del frontend Angular.

Alcance del documento

Este documento describe los requerimientos funcionales y no funcionales del sistema en un nivel intermedio: suficiente para entender que hace cada modulo y cuales son las reglas de negocio principales, sin entrar en detalle de implementacion de clases o codigo fuente.

2. Arquitectura general

Vision de alto nivel

El sistema sigue una arquitectura de microservicios. El frontend Angular se comunica exclusivamente con el API Gateway (puerto 4040), que enruta las peticiones hacia los microservicios registrados en Eureka. Los servicios principales son: ms-auth (autenticacion JWT), pos-service (logica central del POS) y ms-clientes (gestion de clientes en base de datos separada).

Componentes de infraestructura

API Gateway (Spring Cloud Gateway): punto de entrada unico, filtro JWT, CORS.

Registry Server (Eureka): descubrimiento de servicios en puerto 8761.

Config Server: configuracion centralizada desde repositorio Git (Config/).

PostgreSQL pos_inventory (puerto 5433): datos del POS (productos, ordenes, caja).

PostgreSQL clientes_db (puerto 5434): datos de clientes.

Frontend Angular (puerto 4200): interfaz web, no dockerizada.

Flujo de comunicacion

1. El usuario accede al frontend en `http://localhost:4200`.
2. Tras login, el token JWT se guarda en `sessionStorage`.
3. Todas las peticiones al backend pasan por `http://localhost:4040`.
4. El Gateway valida el token contra ms-auth antes de reenviar a pos-service o ms-clientes.
5. pos-service puede consultar ms-clientes via OpenFeign al crear ventas.

Rutas del Gateway

`/auth-server/auth/**` -> ms-auth (publico para login/register).

`/pos-venta-service/**` -> pos-service (requiere JWT).

`/clientes-service/**` -> ms-clientes (requiere JWT).

3. Requerimientos funcionales - Autenticacion

RF-AUTH-01: Registro de usuarios

El sistema debe permitir registrar nuevos usuarios con nombre de usuario y contraseña. La contraseña se almacena hasheada con BCrypt en la tabla user_info de la base de datos pos_inventory.

RF-AUTH-02: Inicio de sesion

El sistema debe autenticar usuarios mediante POST /auth-server/auth/login. Si las credenciales son validas, retorna un accessToken JWT con expiracion de aproximadamente 1 hora.

RF-AUTH-03: Proteccion de rutas

Todas las rutas del frontend excepto /login y /register deben estar protegidas. El authGuard redirige a login si no hay token. El authInterceptor adjunta Authorization: Bearer en cada peticion HTTP y cierra sesion ante 401.

RF-AUTH-04: Listado de usuarios registrados

La pantalla /accounts muestra los usuarios registrados en el servicio de autenticacion (no confundir con cuentas de cajero del POS).

Credenciales iniciales

El sistema incluye un usuario seed: admin / admin123 para pruebas iniciales.

4. Requerimientos funcionales - Punto de venta

RF-POS-01: Interfaz de venta

La pantalla /pos presenta categorias, productos filtrados por categoria, un carrito de compra local y selector de cliente. El operador agrega productos al carrito ajustando cantidades antes de confirmar la venta.

RF-POS-02: Tipos de orden

Cada venta debe clasificarse con uno de estos tipos:

SALE (venta normal), CREDIT (credito), COURTESY (cortesía), DAMAGED (producto danado). El tipo se envia al backend al crear la orden.

RF-POS-03: Asociacion de cliente

Toda venta debe asociarse a un cliente. Por defecto se usa el cliente id=1 ('Clientes Varios'). El operador puede buscar y seleccionar otro cliente desde el microservicio ms-clientes.

RF-POS-04: Validacion de caja abierta

No se permite registrar una venta si no existe una caja en estado OPEN. El frontend valida via CashBoxStateService y el backend rechaza la operacion en OrderServiceImpl si no hay caja abierta.

RF-POS-05: Calculo automatico de totales

El backend obtiene el precio unitario de cada producto, calcula subtotal (precio x cantidad) por linea y suma el total de la orden automaticamente.

RF-POS-06: Numeracion de ordenes

Cada orden recibe un numero secuencial por sesion de caja (cash box). El numero se incrementa automaticamente al crear nuevas ventas en la misma caja.

5. Requerimientos funcionales - Gestion de caja

RF-CAJA-01: Apertura de caja

El operador selecciona una cuenta de cajero (tabla account) e ingresa un saldo inicial. El sistema crea un registro en general_cash_box con estado OPEN.

RF-CAJA-02: Caja unica abierta

Solo puede existir una caja abierta a la vez. El sistema impide abrir otra caja mientras haya una sesion OPEN activa.

RF-CAJA-03: Cierre de caja

Al cerrar, el sistema calcula el saldo final con la formula:

$\text{saldo_final} = \text{saldo_inicial} + \text{total_ventas} + \text{ingresos} - \text{egresos}$.

La caja pasa a estado CLOSED y ya no acepta ventas ni movimientos.

RF-CAJA-04: Historial de sesiones

La pantalla /cash-box muestra el historial de sesiones de caja con sus saldos, fechas y estados.

RF-CAJA-05: Estado compartido en frontend

CashBoxStateService mantiene sincronizada la caja abierta entre el layout principal, POS, ordenes y movimientos de caja.

6. Requerimientos funcionales - Movimientos de caja

RF-MOV-01: Registro de ingresos y egresos

La pantalla /cash-movements permite registrar movimientos manuales de tipo INCOME (ingreso) o EXPENSE (egreso) asociados a la caja abierta.

RF-MOV-02: Restriccion por estado de caja

Los movimientos solo pueden registrarse mientras la caja este OPEN. El backend valida esta condicion en CashMovementServiceImpl.

RF-MOV-03: Impacto en cierre

Los ingresos y egresos registrados se incluyen en el calculo del saldo final al cerrar la caja.

7. Requerimientos funcionales - Catalogo

RF-CAT-01: Gestion de categorias

CRUD completo de categorias en /categories. Cada categoria puede tener nombre, descripcion e imagen asociada almacenada en la base de datos (BYTEA).

RF-CAT-02: Gestion de productos

CRUD completo de productos en /products. Campos: nombre, descripcion, precio, categoria, imagen. Soporte de busqueda por nombre y filtro por categoria.

RF-CAT-03: Imagenes en base de datos

Las imagenes de productos y categorias se suben via multipart/form-data y se almacenan como BYTEA en PostgreSQL. Tamano maximo: 2 MB por imagen.

RF-CAT-04: Visualizacion en POS

El punto de venta muestra miniaturas de categorias y productos. Las imagenes se descargan como blob autenticado y se cachean en memoria en el frontend.

Limitacion conocida

El sistema no implementa control de stock ni inventario. Los productos son referencias de catalogo (nombre, precio, categoria) sin cantidades disponibles.

8. Requerimientos funcionales - Clientes

RF-CLI-01: Microservicio independiente

Los clientes se gestionan en ms-clientes con base de datos propia (clientes_db). Esto permite separar el dominio de clientes del nucleo POS.

RF-CLI-02: CRUD de clientes

La pantalla /clients permite listar, buscar y crear clientes con campos: nombre completo, tipo de documento, numero de documento, direccion y correo.

RF-CLI-03: Busqueda

Busqueda por numero de documento (DNI/RUC) o por nombre parcial con debounce.

RF-CLI-04: Cliente por defecto

Existe un cliente seed id=1 'Clientes Varios' usado cuando no se selecciona cliente o cuando falla la integracion Feign entre pos-service y ms-clientes.

RF-CLI-05: Snapshot en ventas

Al crear una orden, el backend guarda customer_name y customer_document como snapshot en la orden, preservando los datos aunque el cliente cambie despues.

9. Requerimientos funcionales - Ordenes y reportes

RF-ORD-01: Historial de ordenes

La pantalla /orders lista las ventas. Si hay caja abierta, filtra por esa caja; si no, muestra todas las ordenes.

RF-ORD-02: Detalle de orden

Mediante un dialogo se visualizan las lineas de detalle (producto, cantidad, precio unitario, subtotal) de cada orden.

RF-ORD-03: Eliminacion de ordenes

El sistema permite eliminar ordenes. La operacion esta restringida si la caja asociada ya esta cerrada.

RF-REP-01: Reporte por fecha

La pantalla /reports consulta ordenes y total de ventas para una fecha seleccionada mediante GET /order/report/list y GET /order/report/total.

RF-DASH-01: Panel principal

El dashboard (/dashboard) muestra resumen de caja abierta y total de ventas del dia actual.

10. Requerimientos funcionales - Cuentas de cajero

RF-ACC-01: Gestion de cuentas POS

Las cuentas de cajero (tabla account) son distintas de los usuarios JWT (tabla user_info). Representan operadores vinculados a sesiones de caja.

RF-ACC-02: CRUD de cuentas

El backend expone CRUD en /pos-venta-service/account con email unico por cuenta.

RF-ACC-03: Uso en apertura de caja

Al abrir caja, el operador selecciona una cuenta de cajero existente. Esta cuenta queda asociada a la sesion general_cash_box.

11. Reglas de negocio criticas

RN-01: Caja obligatoria para ventas

Toda operacion de venta requiere una caja en estado OPEN. Sin caja abierta, el frontend muestra advertencia y el backend retorna error.

RN-02: Una sola caja abierta

No se puede abrir una segunda caja mientras exista una sesion OPEN.

RN-03: Numeracion secuencial

Los numeros de orden son secuenciales dentro de cada sesion de caja.

RN-04: Snapshot de cliente

Los datos del cliente se copian a la orden al momento de la venta.

RN-05: Calculo de totales en backend

Los precios y totales se calculan en el servidor, no se confia en valores enviados por el cliente para el total final.

RN-06: Arqueo de caja

$\text{saldo_final} = \text{saldo_inicial} + \text{ventas} + \text{ingresos} - \text{egresos}$.

RN-07: Bloqueo post-cierre

Ordenes y movimientos no pueden modificarse/eliminarse si la caja esta CLOSED.

RN-08: Resiliencia Feign

Si ms-clientes no responde al crear una venta, se usa el cliente por defecto.

12. Requerimientos no funcionales

RNF-01: Despliegue con Docker

El backend se despliega via docker-compose en ms/ con healthchecks, seeds SQL automaticos y variables de entorno en ms/.env.

RNF-02: Seguridad JWT

Tokens HMAC-SHA con secret configurable (minimo 32 caracteres). Contraseñas hasheadas con BCrypt. Gateway valida token antes de reenviar.

RNF-03: CORS

Origenes permitidos configurables via variable CORS_ORIGIN (default localhost:4200).

RNF-04: Documentacion API

pos-service expone anotaciones Swagger/OpenAPI (@Tag, @Operation) en controladores.

RNF-05: Descubrimiento de servicios

Microservicios registrados en Eureka para balanceo lb:// via Gateway.

RNF-06: Sin control de roles

No existe RBAC: cualquier usuario autenticado accede a todos los endpoints protegidos. No hay @PreAuthorize ni roles diferenciados.

RNF-07: Requisitos de hardware

Minimo 8 GB RAM, ~5 GB disco, puertos 4040, 5433, 5434, 4200 libres.

13. Stack tecnologico y limitaciones

Backend

Spring Boot 4.0.5, Spring Cloud 2025.1.1, Java 21, JPA/Hibernate, OpenFeign, JWT (jjwt), PostgreSQL 15/16, Maven.

Frontend

Angular 20.3, standalone components, Angular Material, signals + RxJS, lazy loading, HttpClient con interceptor funcional.

Infraestructura

Docker Compose, Spring Cloud Gateway, Netflix Eureka, Spring Cloud Config.

Limitaciones del alcance actual

Sin control de inventario/stock.

Imagenes limitadas a 2 MB almacenadas en BD (no en filesystem/S3).

Sin roles ni permisos granulares.

Sin facturacion electronica ni integracion con SUNAT.

Sin modo offline ni sincronizacion PWA.

Frontend no dockerizado (requiere Node.js 20+ para desarrollo).

Pantallas del frontend

/login, /register, /dashboard, /pos, /orders, /cash-box, /cash-movements, /products, /categories, /clients, /accounts, /reports.